

【新·拯救者】数据库概论(原理)速成

(期末+复试+专升本+二级均可) ©无忌教育



期末速成

/

考研复试基础速成

/

专升本均合适

/

自考等



适用于所有 数据库原理 课本



适用于所有数据库

概论 课本



【新拯救者】Ep0_前言

这里讲的是【🌷速成🌷速成🌷速成】版本，抽取重点考的题型，翻译为人话!!!💖

👉特色:

- 1 先讲本章考点,紧接着
- 2 接着就讲对应的习题

 **好评:**

拟录取了

真tm顶

这钱花的值

信任很重要

12:27

真的假的啊

这么厉害吗 直接上岸啊

那必须

211

忘了还愿了现在还得来应及该，数据上听课不零基础只看成了速成课72分还愿，大学期末绰绰有余！兄弟们冲啊我计算机系已经决定大学四年所有的课都靠老师了！讲得真的是我所有看过的速成课里最好的，所有类型的课！听明白了简洁一讲就会，根本不费脑子！[Cat2_Nyam

其他科的时候想来还愿[脱单doge]，上学期没学过Java，极限两天一晚复习，期末八十多分，全靠自大学，准备入手数据库结构[上机]

回复 @超爱山山:同乐，我也过了

回复 @超爱山山 :同乐，我也过了

 文档下载:



手机/平板/电脑打开---"课程主页"---"资料", 下载完记得打开【大纲模式】（这样就会跟我一样有目录了）

不会打开大纲模式?

教程: <https://bbs.csdn.net/topics/609294605>

开始大家不是很熟悉，别着急

当你二刷时，只会感叹：

【回首看，已如古人】~

让我们一起出征【数据库速成】吧，爱谁谁，都多余了哦~~~

👂👂👂 ps:目录为我讲课顺序自己制作，

别说你的目录跟我书的不一样，我就不学了，考的通用内容是一样的哦！

期末速成 / 考研复试基础速成 / 专升本均合适



适用于所有 数据库原理 课本



适用于所有数据库

概论 课本

【新拯救者】Ep1_绪论

1.数据库系统的基本概念

1.1 数据 (Data)

- 定义：描述事物的符号记录，是数据库中存储的基本对象
- 示例：学生表中的学号（2025001）、姓名（张无忌）、年龄（30）等字段值

1.2 数据库 (DB)

- 定义：长期存储在计算机内、有组织的、可共享的数据集合
- 示例：学校教务管理系统包含学生表、课程表、选课关系表构成的数据库

1.3 数据库管理系统 (DBMS)

- 定义：管理数据库的软件系统，提供 ①数据库定义功能；②数据存取功能；③数据库运行管理；④数据库的建立和维护功能等功能
- 示例：MySQL实现创建表结构（CREATE TABLE）、执行SQL查询（SELECT）等操作

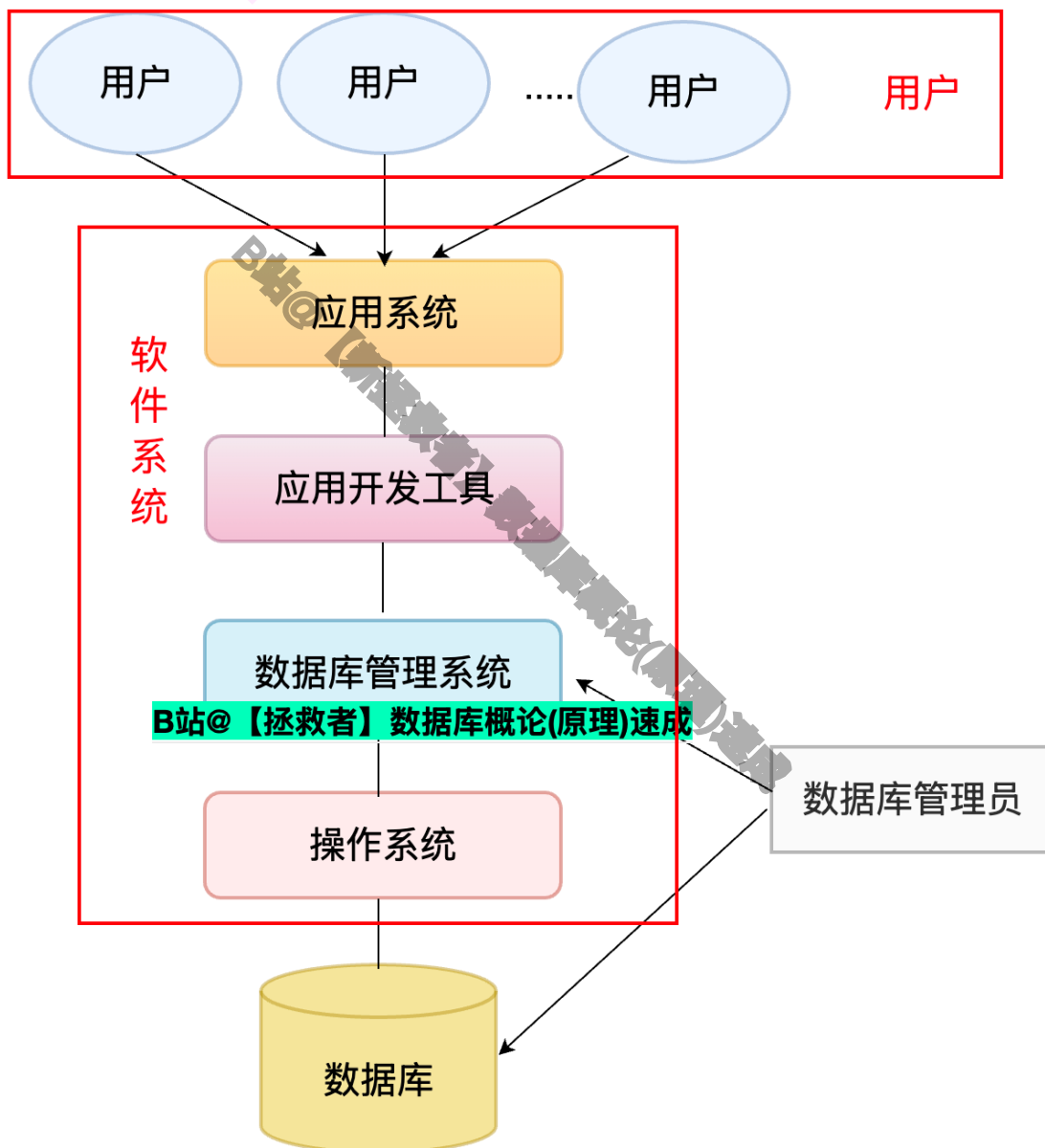
1.4 数据库系统 (DBS)

- 组成: 数据库 + DBMS + 应用程序 + 用户(使用DB的人)



无忌教育官网 | wuji-edu.com

- 示例: 电商平台 (应用) 通过Java程序调用Oracle DBMS 管理订单、用户等数据



2. 数据库的发展阶段

2.1 人工管理阶段 (1945-1955)

- 特点：无直接存储设备，数据依赖程序，无共享性。
- 示例：早期科学计算中，数据随程序一起提交。

2.2 文件系统阶段（1955-1965）

- 特点：磁盘存储出现，数据以文件形式组织，但冗余度高。
- 示例：企业用文本文件存储员工信息，易产生重复数据。

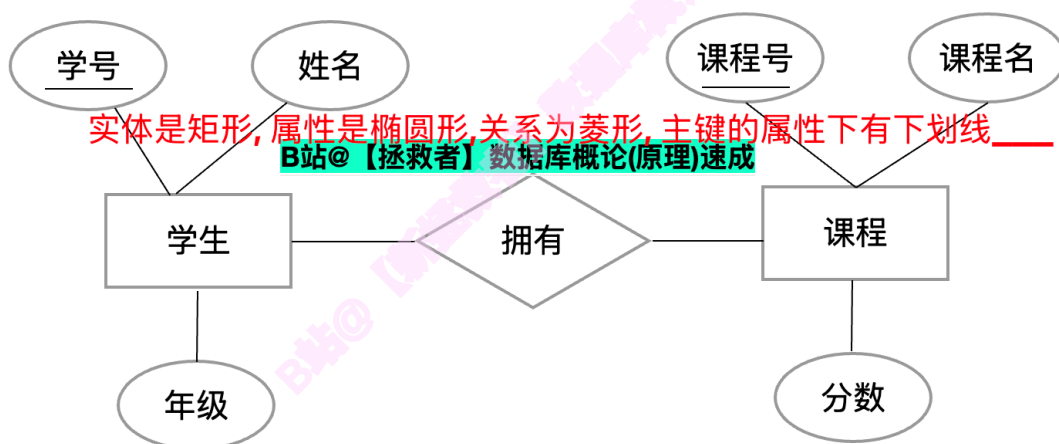
2.3 数据库系统阶段（1965至今）

- 特点：数据按模型组织，共享性高，独立性增强。
- 示例：关系型数据库MySQL支持多用户并发访问同一数据表。

3. 数据模型

3.1 概念模型（E-R模型）

- 实体（Entity）：客观存在的事物（如“学生”）。
- 属性（Attribute）：实体特征（如“学号”“姓名”）。
- 联系（Relationship）：实体间关系（如“选修”课程）。
- 示例：学生（学号，姓名，年级）与课程（课程号，课程名，分数）通过“选修”联系。



3.2 逻辑模型

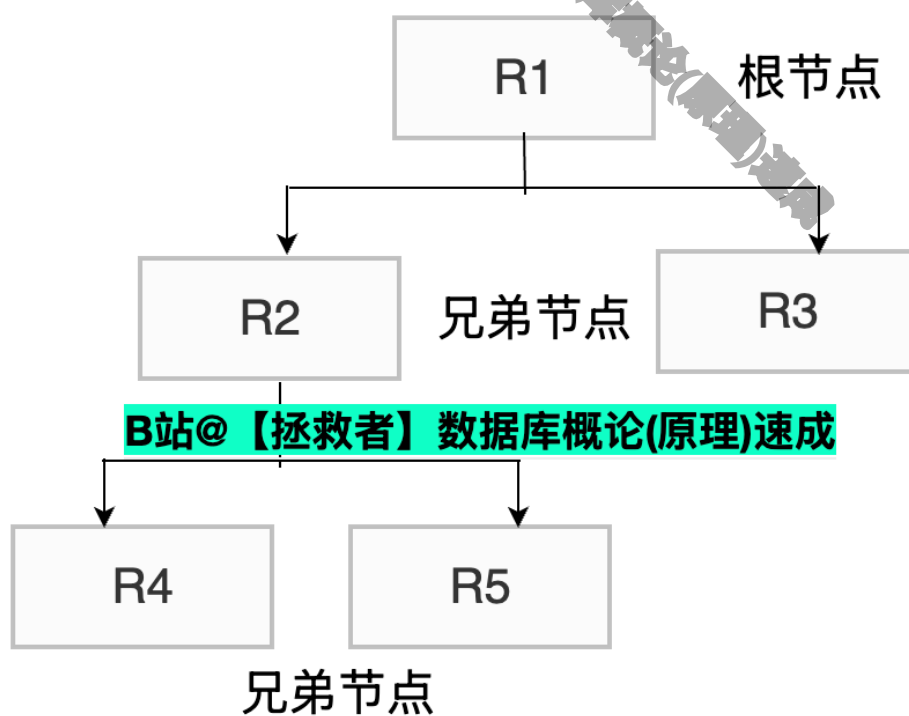
- **层次模型**：树状结构，适合简单层级关系（如家族谱）



无忌教育官网 | wuji-edu.com

条件：

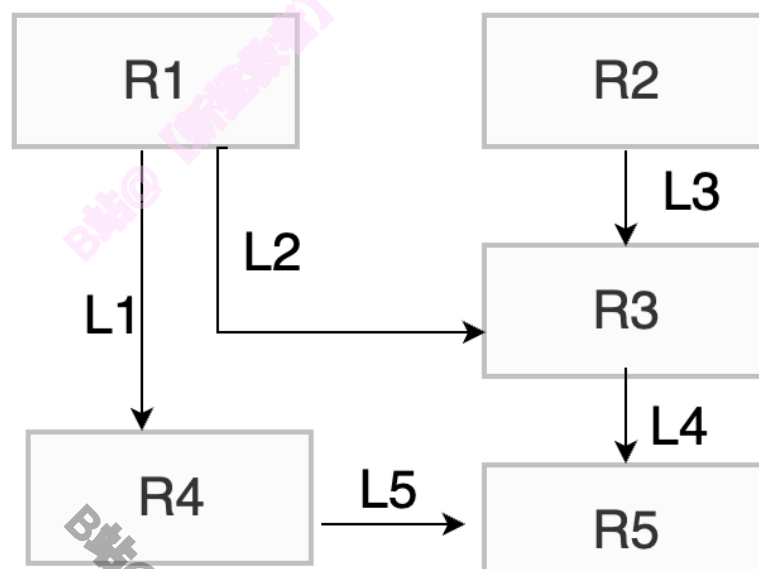
- 1、有且只有一个结点没有双亲结点，这个结点称为根结点。
- 2、根以外的其他结点有且只有一个双亲结点。



- **网状模型**：复杂网络结构，支持多对多关系（如交通网络）

条件

- 1.允许一个以上的结点无双亲。
- 2.一个结点可以有多个双亲。



B站@【拯救者】数据库概论(原理)速成

• 关系模型：二维表结构 后续要讲的重点

id	username	password	nickname	mobile
# bigint	varchar(64)	varchar(64)	varchar(64)	varchar(20)
4	猪猪	123	会飞的猪	15866669999
6	星空	123123	云边快乐兔	15853469999
7	张三	555	武当掌门人	18899908888
8	无忌科技	123	嘻嘻	13344456667
19	小白	123456	白哥	15866668888
20	李四	123`	飞翔的QQ	1345678990
21	测试	1111	自由自在	1234455566

关系： 一个关系对应通常说的一张表。

元组： 表中的一行。

属性： 表中的一列。

码： 也称码键(主键)，唯一标识实体的属性，是整个实体集的性质,包括超码，候选码，主码。

域： 是一组具有相同数据类型的值的集合。

分量： 元组中的一个属性。

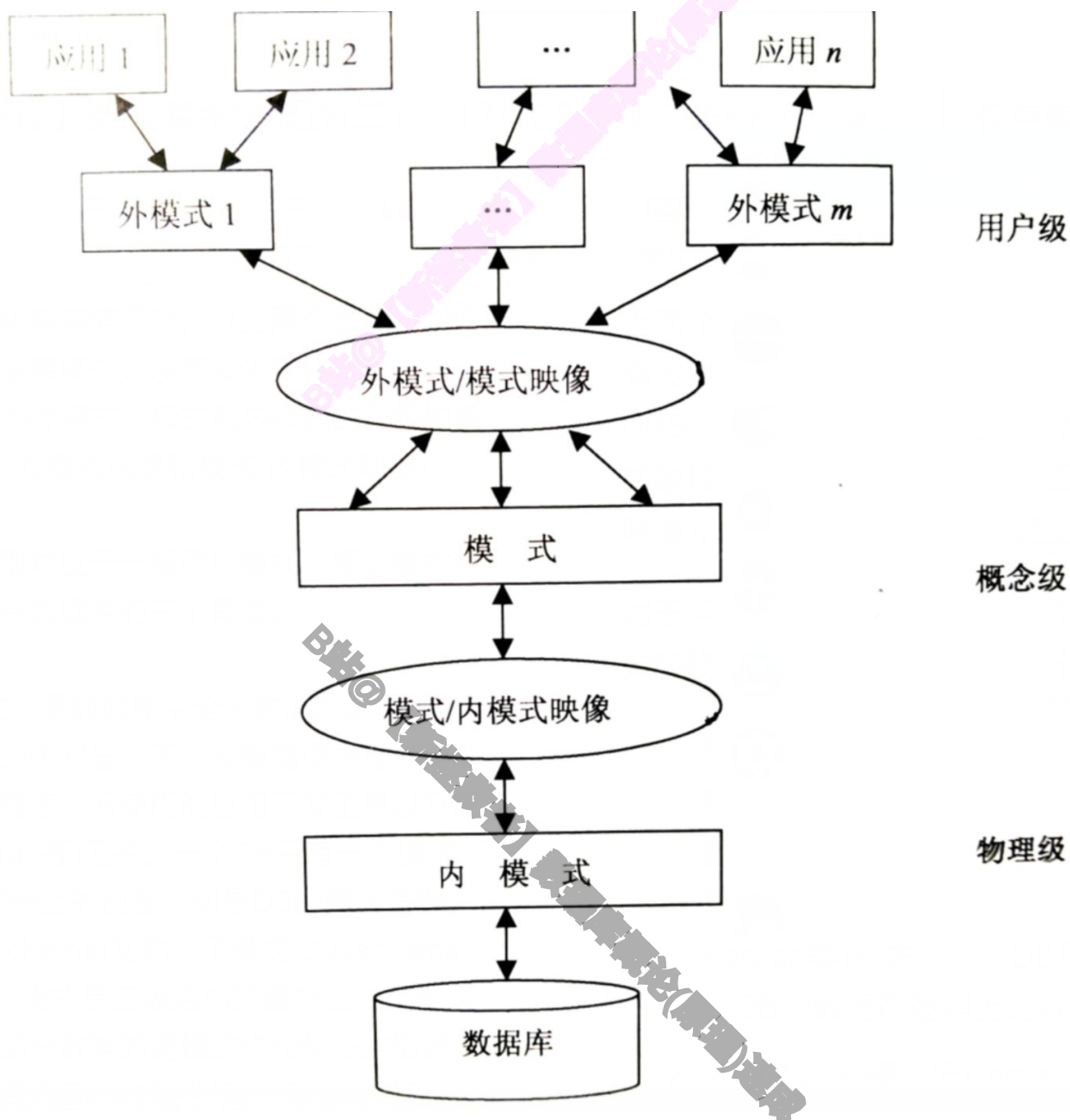
3.3 物理模型

- 描述数据在磁盘上的存储方式（如B+树索引、哈希存储）,是最底层的抽象。

四、数据库系统结构

模式数据库中全体数据的逻辑结构和特征的描述,它仅仅涉及型的描述,不涉及具体的值。

某一个具体值称为模式的一个实例



4.1 三级模式结构

- **外模式:** 也叫用户视图或子模式,他是包括应用程序员和最终用户能够看见和使用的局部数据的逻辑结构和特征的描述

如 学生仅见“课程号、成绩”

- **模式:** 也称为逻辑模式,是数据库中全体数据的逻辑结构和特征的描述

如包含“学生、课程、选课”表

- **内模式**：也称为存储模式,一个数据库只有一个内模式,他是数据库物理表示

如以什么存储方式(顺序存储, B+树存储)来存储数据、索引按照什么方式组织、数据是否压缩、数据是否加密

4.2二级映像

数据库系统在三级模式之间提供了两级映像：模式/内模式的映像、外模式/模式的映像。这两级映射保证了数据库中的数据具有较高的物理独立性和逻辑独立性。

- **外模式/模式映像**：保证逻辑独立性

用户的应用程序与数据库结构是相互独立的。数据的逻辑结构发生变化后,用户程序也可以不修改

原始的学生逻辑结构为【学号,姓名,性别】,现在添加新的属性【出生日期】,逻辑结构变更为【学号,姓名,性别,出生日期】。后续由DBA对各个外模式/模式映像作相应改变,这一映像功能保证了数据的局部逻辑结构不变(即外模式保持不变)。又由于应用程序是依据外模式编写的,所以应用程序不必修改,从而保证了数据与程序间的逻辑独立性。

- **模式/内模式映像**：保证物理独立性

当数据库的内模式发生改变时,数据的逻辑结构不变。由于应用程序处理的只是数据的逻辑结构,这样物理独立性可以保证,当数据的物理结构改变了,应用程序不用改变。
(如更换存储设备无需修改SQL语句)。

5.数据库系统的组成

- 1、**硬件平台及数据库**: 足够大内存、磁盘或磁盘阵列等设备,较高的通道能力以提高数据的传送率
- 2、**软件**: 数据库系统的软件主要包括:
 - DBMS。DBMS是为数据库的建立、使用和维护配置的系统软件。
 - 支持DBMS运行的**操作系统**。
 - 具有与**数据库接口的高级语言及编译系统**,便于开发应用程序。
 - 以DBMS为核心的**应用开发工具**。
 - 为特定应用环境开发的**数据库应用系统**。
- 3、**人员**: 开发、管理和使用数据库的人员主要包括数据库管理员、系统分析员和数据库设计人员、应用程序员和用户。

数据库管理员职责： 全面负责数据库系统的日常管理、维护和运行的人员

系统分析员、数据库设计人员： 从事需求分析、信息系统项目架构设计（包括概要设计和详细设计）、开发阶段的主要模块的规划、设计和测试

应用程序员： 负责设计和编制应用程序的人员。他们通常使用C语言、数据库语言等来设计和编写应用程序，

用户： 用户是管理、开发、使用数据库的主体

6.数据库系统类型与应用

6.1类型

- **关系型数据库：** 如MySQL、Oracle，适合结构化数据。
- **非关系型数据库：** 如MongoDB（文档型）、Redis（键值型），适合非结构化数据。
- **分布式数据库：** 如Cassandra，支持海量数据与高并发访问



无忌教育官网 | wuji-edu.com

【新拯救者】Ep2_绪论习题



在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep3_关系数据库(上)

1.关系模式

关系模式：对关系的描述,它定义了数据库中表的结构，包括表的名称、列的名称及数据类型、列之间的关系以及约束条件等

关系：关系是一张表，一张二维表

$R(U, D, DOM, F)$

R:关系名

U:所有属性名

D: 属性来自哪些域

DOM: 属性和域的映射

F:属性间的依赖关系

患者（患者编号，患者姓名，性别，出生日起，所在单位）

2.关系操作

插入、查询、删除、修改

其中查询操作分为：（选择、投影、并、差、笛卡尔积）

3.关系语言分类

关系代数语言(重点) 关系演算语言(不重要) SQL语言(后面章重点讲)

4.关系代数语言

关系代数是一种抽象的查询语言, 它用对关系的运算来表达查询。

三大要素: 运算对象(关系)、运算符(集合运算符和专门的关系运算符) 和运算结果(关系)

集合运算符: 并($\cup$)、差($-$)、交($\cap$)、笛卡尔积($\times$)

专门的关系运算符: 选择(σ)、投影(π)、连接($\bowtie$)、除($\div$)

算术比较符: 大于($>$)、大于等于($\geq$)、小于($<$)、小于等于($\leq$)、等于($=$)、不等于($\neq$)

逻辑运算符: 非($\neg$)、与($\wedge$)、或($\vee$)

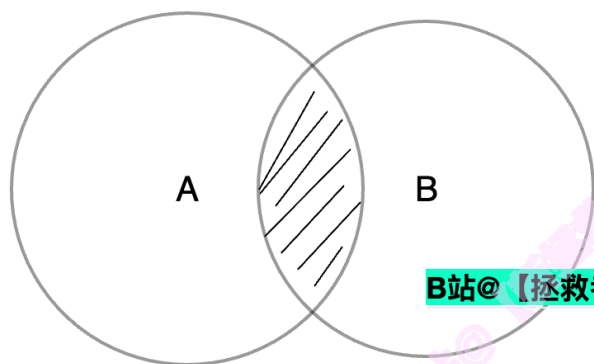
❤️补充❤️

5种基本运算: 并, 差, 笛卡儿积, 选择, 投影

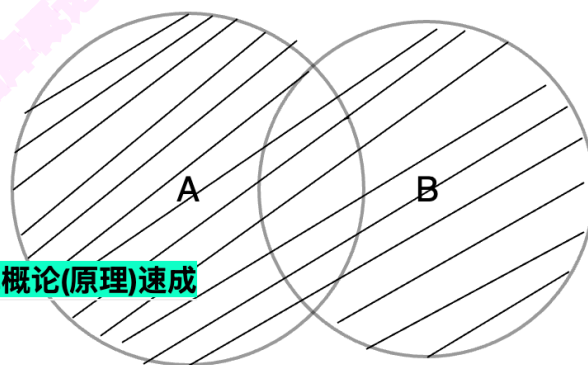


无忌教育官网 | wuji-edu.com

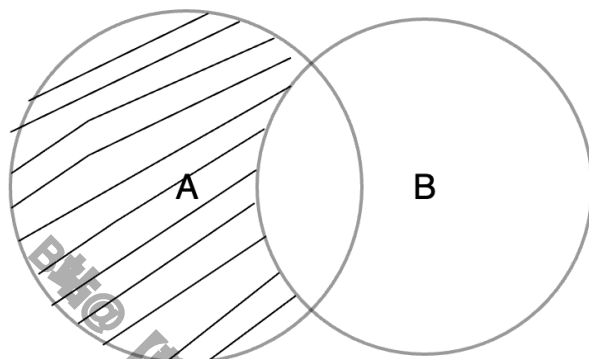
集合运算符:



$A \cap B$



$A \cup B$



$A - B$

a1	a2
a	aa
c	cc

b1	b2
b	bb
d	dd

笛卡尔积 X

a1	a2	b1	b2
a	aa	b	bb
a	aa	d	dd
c	cc	b	bb
c	cc	d	dd

wuji-edu.com

R		
A	B	C
a ₁	b ₁	c ₁
a ₁	b ₂	c ₂
a ₂	b ₂	c ₁

(a)

S		
A	B	C
a ₁	b ₂	c ₂
a ₁	b ₃	c ₂
a ₂	b ₂	c ₁

(b)

R ∪ S		
A	B	C
a ₁	b ₁	c ₁
a ₁	b ₂	c ₂
a ₂	b ₂	c ₁
a ₁	b ₃	c ₂

(c)

R ∩ S		
A	B	C
a ₁	b ₂	c ₂
a ₂	b ₂	c ₁

(d)

R × S					
R.A	R.B	R.C	S.A	S.B	S.C
a ₁	b ₁	c ₁	a ₁	b ₂	c ₂
a ₁	b ₁	c ₁	a ₁	b ₃	c ₂
a ₁	b ₁	c ₁	a ₂	b ₂	c ₁
a ₁	b ₂	c ₂	a ₁	b ₂	c ₂
a ₁	b ₂	c ₂	a ₁	b ₃	c ₂
a ₁	b ₂	c ₂	a ₂	b ₂	c ₁
a ₂	b ₂	c ₁	a ₁	b ₂	c ₂
a ₂	b ₂	c ₁	a ₁	b ₃	c ₂
a ₂	b ₂	c ₁	a ₂	b ₂	c ₁

(f)

R - S		
A	B	C
a ₁	b ₁	c ₁

(e)

并: $R \cup S$ 有相同属性, 去重

差: $R - S$ 有相同的属性, 留下在R不在S中的元组

交: $R \cap S$ 有相同的属性

笛卡尔积: $R \times S = \{\widehat{t_r t_s} \mid t_r \in R \wedge t_s \in S\}$ 结果共有 $k_1 \times k_2$ 个元组, $(n + m)$ 个属性

专门的关系运算符:

选择(σ)

Stu

学号	姓名	年龄
001	张无忌	11
002	张无鸡	22
003	张无坤	33

$\sigma_{\text{年龄}='11'}(\text{Stu})$

学号	姓名	年龄
001	张无忌	11

投影(π)

Stu

学号	姓名	年龄
001	张无忌	11
002	张无鸡	22
003	张无坤	33

π 姓名,年龄 (Stu)

姓名	年龄
张无忌	11
张无鸡	22
张无坤	33

【新拯救者】Ep4_关系数据库(下)

连接 (⋈)

✕自然连接: 把共同属性进行等值连接

悬浮元组: 两个关系R和S在自然连接时, 关系R和S中被舍弃的元组

外连接: 把悬浮元组舍弃的元组也保存在结果关系中, 而在其他属性上填空值。

左外连接: 左连接从左表取出所有记录, 与右表匹配。如果没有匹配, 以null值代表右边表的列

右外连接: 从右表取出所有记录, 与左表匹配。如果没有匹配, 以null值代表左边表的列

c_id # int	c_name varchar(255)	t_id # int	t_id # int	t_name varchar(255)
1	Java 速成	1	1	张无忌
2	C 语言速成	2	2	张无坤
3	Python 速成	3	3	阿坤
4	SSM 速成	5	4	阿鸡
			6	菜菜老师

course

teacher

自然连接

c_id	c_name	t_id	t_id(1)	t_name
1	Java 速成	1	1	张无忌
2	C 语言速成	2	2	张无坤
3	Python 速成	3	3	阿坤

左外连接

左外连接



右外连接



c_id	c_name	t_id	t_id(	t_name
1	Java 速成	1	1	张无忌
2	C 语言速成	2	2	张无坤
3	Python 速成	3	3	阿坤
4	SSM 速成	5	(NUL	(NULL)

右外连接

c_id	c_name	t_id	t_id(	t_name
1	Java 速成	1	1	张无忌
2	C 语言速成	2	2	张无坤
3	Python 速成	3	3	阿坤
(NUL	(NULL)	(NUL	4	阿鸡
(NUL	(NULL)	(NUL	6	菜菜老师

除(÷)

保留A中满足B的,而且A中列要去掉B的列

姓名	选修课程	选修课程	姓名
张无忌	计算机	计算机	张无忌
张无忌	数据库	数据库	
张无坤	数据库		
张无坤	拯救者系列课程		
赵四	计算机		
赵四	数据库		赵四

5.关系的完整性

1. 实体完整性 (主键不可为空且唯一)
2. 参照完整性 (外码为空或者为别的表的主码)
3. 用户定义的完整性
自定义业务约束 (如年龄>0)

【新拯救者】Ep5_关系数据库习题

   在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep6_SQL语言 (上)

SQL (Structured Query Language) 是关系数据库的标准操作语言,其功能包括数据库模式创建、数据库数据的插入与修改、数据库数据的查询、数据库安全性定义与控制等诸多功能

SQL特点

1. 综合统一

集数据定义语言（DDL），数据操纵语言（DML），数据控制语言（DCL）功能于一体。数据操作符统一。

2. 高度非过程化

非关系数据模型的数据操纵语言“面向过程”，必须制定存取路径。

SQL只要提出“做什么”，无须了解存取路径。存取路径的选择以及SQL的操作过程由系统自动完成。

3. 面向集合的操作方式

非关系数据模型采用面向记录的操作方式，操作对象是一条记录。操作起来较为复杂繁琐，每次查询耗时耗力。

SQL采用集合操作方式，即操作对象、查找结果可以是元组的集合，一次插入、删除、更新操作的对象可以是元组的集合。

4. 以同一种语法结构提供多种使用方法

SQL是独立的语言，能够独立地用于联机交互的使用方式。

SQL又是嵌入式语言，SQL能够嵌入到高级语言（例如C，C++，Java）程序中，供程序员设计程序时使用。

5. 语言简洁，易学易用

SQL功能极强，但由于设计巧妙，语言十分简洁

SQL的数据类型

数据类型	含义
CHAR(n), CHARACTER(n)	长度为 n 的定长字符串
VARCHAR(n), CHARACTERVARYING(n)	最大长度为 n 的变长字符串
CLOB	字符串大对象
BLOB	二进制大对象
INT, INTEGER	长整数 (4 字节)
SMALLINT	短整数 (2 字节)
BIGINT	大整数 (8 字节)

数据类型	含义
NUMERIC(p, d)	定点数, 由 p 位数字 (不包括符号、小数点) 组成, 小数点后面有 d 位数字
DECIMAL(p, d), DEC(p, d)	同 NUMERIC
REAL	取决于机器精度的单精度浮点数
DOUBLE PRECISION	取决于机器精度的双精度浮点数
FLOAT(n)	可选精度的浮点数, 精度至少为 n 位数字
BOOLEAN	逻辑布尔量
DATE	日期, 包含年、月、日, 格式为 YYYY-MM-DD
TIME	时间, 包含小时、分钟、秒, 格式为 HH:MM:SS
TIMESTAMP	时间戳类型
INTERVAL	时间间隔类型

DDL (Data Definition Language) : 定义数据库结构



无忌教育官网 | wuji-edu.com

创建数据库/表

CREATE DATABASE 数据库名;

CREATE TABLE 表名 (

```
    列名 数据类型 [约束],
    ...
);
```

约束名

NOT NULL	限制列取值非空
DEFAULT	指定列的默认值
UNIQUE	限制列取值不能重复
CHECK	限制列的取值范围
PRIMARY KEY	指定表的主键
FOREIGN KEY	指定表的外键

-- 创建学生表

```
CREATE TABLE Student (
    s_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (Age >= 18),
    major VARCHAR(250) DEFAULT '编程'
);
```

-- 创建课程表

```
CREATE TABLE Course (
    c_id VARCHAR(10) PRIMARY KEY,
    c_name VARCHAR(50) UNIQUE
);
```

修改表结构

```
ALTER TABLE 表名 ADD/DROP/MODIFY COLUMN 列名 数据类型  
[约束];
```

-- 添加邮箱列

```
ALTER TABLE Student ADD email VARCHAR(100) UNIQUE;
```

-- 修改列类型

```
ALTER TABLE Student MODIFY age SMALLINT;
```

-- 删除列

```
ALTER TABLE Student DROP COLUMN email;
```

删除对象

删库跑路: `DROP DATABASE 数据库名;`

删表: `DROP TABLE 表名 [RESTRICT | CASCADE];`

`CASCADE`呢 是如果表有外键,视图 触发器的话,也会强行删除,
`RESTRICT`恰恰相反哈

【新拯救者】Ep7_SQL语言 (中)

DML (Data Manipulation Language) : 操作表中数据

SELECT查询

SELECT 列名 FROM 表名

[WHERE 条件]

[GROUP BY 分组列]

[HAVING 分组条件] 依据分组的选择条件对组进行过滤（与 GROUP BY搭配使用）。

[ORDER BY 排序列] ASC表示将结果按升序排序，DESC表示按降序排序（系统默认为ASC）

-- 简单查询

SELECT * FROM Student;

SELECT name, age FROM Student;

--查询给列起别名 给表起别名也是一样

SELECT * FROM Student s;

SELECT name n, age a FROM Student;

-- 结果去重

加上DISTINCT关键自己就好

-- 条件查询

查询条件	谓词
比较	=, >, <, >=, <=, !=, <>, !>, !<; NOT+上述比较运算符
确定范围	BETWEEN AND, NOT BETWEEN AND
确定集合	IN, NOT IN
字符匹配	LIKE, NOT LIKE
空值	IS NULL, IS NOT NULL
多重条件（逻辑运算）	AND, OR, NOT

```
SELECT * FROM Student WHERE age BETWEEN 20 AND
```

```
22;
```

-- 模糊查询 _匹配任意一个字符 %匹配0个或多个字符

```
SELECT * FROM Student WHERE Name LIKE '张%';
```

-- 嵌套查询

查询分数高于平均分的学生信息。

```
SELECT student_id, subject, score FROM scores  
WHERE score > (SELECT AVG(score) FROM scores);
```

-- 聚合函数

聚集函数

COUNT(*) 统计元组个数

COUNT([DISTINCT|ALL] 列名) 统计一列中值的个数

sum([DISTINCT|ALL] 列名) 计算一列值的总和 (此列必须为数值型)

AVG([DISTINCT|ALL] 列名) 计算一列值的平均值 (此列必须为数值型)

MAX([DISTINCT|ALL] 列名) 求一列中的最大值

MIN([DISTINCT|ALL] 列名) 求一列中的最小值

```
SELECT major, AVG(age) AS 平均年龄 FROM Student  
GROUP BY major  
HAVING AVG(age) > 21;
```

连接

--自身连接:

一个表与自身连接,为本身这个表起两个别名,然后进行操作

```
SELECT * FROM teacher t1 ,teacher t2 where  
t1.t_id=t2.t_id;
```

t_id # int	t_name varchar(255)		t_id	t_name	t_id(	t_name(
1	张无忌	自己 连接 自己	1	张无忌	1	张无忌
2	张无坤		2	张无坤	2	张无坤
3	阿坤		3	阿坤	3	阿坤
4	阿鸡		4	阿鸡	4	阿鸡
6	菜菜老师		6	菜菜老师	6	菜菜老师

内连接 左外连接 右外连接都讲了

-- 多表查询

```
SELECT s.Name, c.CourseName FROM Student s  
JOIN SC sc ON s.StudentID = sc.StudentID  
JOIN Course c ON sc.CourseID = c.CourseID;
```

带EXISTS 的子查询

not exists 如果后面的子查询没有值,返回1 否则为0
exists 相反

假设表结构如下:

students表: student_id(主键)

courses表: course_id(主键)

SC表: student_id, course_id

从学生表中查询学生选修了全部课程的学生信息

```
SELECT * FROM students s
    WHERE NOT EXISTS ( //当不存在未选修课程时,
说明学生选修了全部课程
        SELECT * FROM courses c
        WHERE NOT EXISTS (
            SELECT * FROM sc //查找该学生未选修的课程
            WHERE sc.student_id = s.student_id AND
sc.course_id = c.course_id
        )
    );
```

带有ANY ALL的子查询

> ANY	大于子查询结果中的某个值
> ALL	大于子查询结果中的所有值
< ANY	小于子查询结果中的某个值
< ALL	小于子查询结果中的所有值
>= ANY	大于等于子查询结果中的某个值
>= ALL	大于等于子查询结果中的所有值
<= ANY	小于等于子查询结果中的某个值
<= ALL	小于等于子查询结果中的所有值
= ANY	等于子查询结果中的某个值
= ALL	等于子查询结果中的所有值 (通常没有实际意义)

集合查询

- UNION 合并多个查询结果, 去重 ,不去重就用UNION ALL
合并两个表的姓名, 去重 (默认)

```
SELECT name FROM employees
UNION
SELECT name FROM managers;
```

- INTERSECT 返回多个查询结果的 交集

```
SELECT A.id FROM table1 A
INNER JOIN table2 B ON A.id = B.id;
```

- EXCEPT 返回第一个查询结果中不在第二个结果中的行
查询计算机科学系的学生与年龄不大于 19岁的学生的差集。

```
SELECT * FROM Student
WHERE major='CS'
EXCEPT
SELECT * FROM Student
WHERE age <=19;
```

【新拯救者】Ep8_SQL语言 (下)

INSERT插入

```
INSERT INTO 表名 (列1, 列2, ...) VALUES (值1, 值2, ...);
```

-- 插入完整记录

```
INSERT INTO Student (s_id, name, age, major)
VALUES (1001, '张无忌', 30, '编程');
```

-- 指定 部分 字段添加值

```
INSERT INTO Student (s_id, name ) VALUES (1001,
'张无忌' );
```

-- 批量 添加值

```
INSERT INTO Student values
( 1001 , 'zhaoliu' , 25 , 'CS' ),
( 1002 , 'liu1' , 26 , 'CS' ),
( 1003 , 'cu02' , 28 , 'edu' );
```

UPDATE更新

```
UPDATE 表名 set 字段名=某个值 [WHERE 条件];
```

-- 更新单列

```
UPDATE Student SET age = 21 WHERE s_id = 1001;
```

-- 更新多列

```
UPDATE Student SET age = 22, name = '张无忌' WHERE
s_id = 1002;
```

DELETE删除

```
DELETE FROM 表名 [WHERE 条件];
```

-- 删除指定记录

```
DELETE FROM Student WHERE s_id = 1002;
```

DCL (Data Control Language) : 权限控制

权限管理

```
GRANT 权限 ON 对象 TO 用户;
```

```
REVOKE 权限 ON 对象 FROM 用户;
```

-- 授予查询权限

```
GRANT SELECT ON Student TO 'user1';
```

-- 撤销插入权限

```
REVOKE INSERT ON Course FROM 'user1';
```



无忌教育官网 | wuji-edu.com

高级功能:

事务控制


```
START TRANSACTION;  
-- DML操作  
COMMIT; -- 或 ROLLBACK;
```

索引

数据量比较大的时候, 查询耗时间长, 建立索引可以有效减少消耗时间

索引可以建立在一列或者多列上

普通索引:

```
CREATE INDEX 索引名 ON 表名 (列名);
```

唯一索引: 索引列的每个值都唯一

```
CREATE UNIQUE INDEX 索引名 ON 表名(列名);
```

主键索引: 是一种特殊的唯一索引, 必须指定为 "PRIMARY KEY"

```
CREATE TABLE 表名 ([...], PRIMARY KEY (列名));
```

组合索引: 可以是单列上创建的索引, 也可以是在多列上创建的索引

```
CREATE TABLE 表名 (  
    列名1 数据类型,  
    列名2 数据类型,  
    列名3 数据类型,  
    INDEX 索引名 (列名1,列名2,列名3));
```

全文索引 (FULLTEXT): 在 CHAR、VARCHAR 或者 TEXT 类型的列上创建。每个表只允许有一个全文索引

```
CREATE FULLTEXT INDEX 索引名 ON 表名 (列名);
```

-- 为频繁查询的列创建索引

```
CREATE INDEX idx_student_name ON Student (name);
```

```
alter INDEX 旧索引名 rename to 新索引名
```

-- 索引名修改

```
alter INDEX AIndex rename to BIndex
```

-- 删除索引

```
DROP INDEX 索引名 ON 表名;
```

视图

也被称作虚表,即虚拟的表,是一组数据的逻辑表示,其本质是对应于一条SELECT结果集被赋予一个名字,即视图名字
视图不存储实际数据



无忌教育官网 | wuji-edu.com

当查询视图时,数据库会动态执行其关联的SQL语句并返回结果。

```
CREATE VIEW 视图名 AS 子查询
```

[WITH CHECK OPTION] 确保通过视图修改的数据始终满足视图的 WHERE 条件。若不满足,操作将被拒绝并报错。

-- 创建年龄>10的学生视图

```
CREATE VIEW my_Students AS
```

```
SELECT s_id, name FROM Student WHERE age >10 ;
```

如果有WITH CHECK OPTION,表示确保通过视图插入或更新的数据必须满足 age > 10 的条件

```
UPDATE my_Students SET age = 9 WHERE s_id = 1;
```

-- 失败 (违反 CHECK OPTION)

-- 使用视图

```
SELECT * FROM CS_Students ;
```

-- 更新视图

视图是不存在的,所以我们对视图的操作,最终要反映到对基本表的操作

```
UPDATE my_Students SET name = 'lisi' WHERE  
s_id = 1;
```

-- 删除视图

```
DROP VIEW 视图名 ;
```

补充

模式定义和删除

```
create schema <模式名> authorization <用户名> [表定义 | 视图定义语句 | 授权定义子句];
```

给用户ZWJ定义一个Learn模式

```
create schema "Learn" authorization 'ZWJ';
```

```
drop schema <模式名> <CASCADE | RESTRICT>
```

删除模式，其中 CASCADE 和RESTRICT 必须二选一，

CASCADE是(级联) 删除模式的同时也把该模式的所有数据库对象删除。

RESTRICT(限制) 如果该模式下有下属对象,比如表视图 就拒绝这个删除语句的执行

【新拯救者】Ep9_SQL语言习题



在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep10_数据库的安全性



无忌教育官网

wuji-edu.com

数据库安全性概述

目的：保护数据库免受未授权访问、篡改或破坏，确保数据的**机密性、完整性和可用性**。

- **身份认证：**验证用户身份。
- **权限控制：**限制用户操作范围。
- **数据保护：**防止数据泄露或损坏。

用户身份认证

验证用户身份是否合法。

实现方式：

1. **用户名/密码：**最常见的认证方式。
2. **多因素认证：**结合密码、短信验证码、生物识别等。

-- 创建用户并设置密码

```
CREATE USER 'user_zwj' IDENTIFIED BY '123456@';
```

权限管理

通过授予或撤销权限，控制用户对数据库对象的访问。

`GRANT`（授权）、`REVOKE`（撤销）、`ROLE`（角色）。

对象类型	对象	操作类型
数据库 模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

1. 授予权限

```
GRANT <权限列表> ON <表名/列名> TO <用户或角色>;
```

```
[with grant option]
```

如果带了with,表示管理员吧权限授予普通用户,然后普通用户获得授权后可以把自己权限授予其他用户

```
-- 授予用户对 students 表的 SELECT 和 UPDATE 权限
```

```
GRANT SELECT,UPDATE ON students TO user_zwj;
```

```
-- 授予用户对 students 表id列的 SELECT 和 UPDATE 权限
```

```
GRANT SELECT,UPDATE ON students(id) TO user_zwj;
```

```
-- 授予所有权限（慎用！）
```

```
GRANT ALL PRIVILEGES ON students TO user_zwj;
```

2. 撤销权限

```
--撤销user_zwj用户students 插入权限
```

```
REVOKE INSERT ON students FROM user_zwj;
```

```
[cascade]
```

3. 角色管理

-- 创建角色并分配权限

```
CREATE ROLE student_reader;
```

```
GRANT SELECT ON students TO student_reade ;
```

-- 将角色赋予用户lihua

```
GRANT student_reader TO lihua;
```

--角色权限收回

revoke 权限 on 表名 from 角色名

```
revoke select on student from student_reader
```

数据加密

目的：防止敏感数据在存储或传输过程中被窃取。

1. 存储加密

(MySQL使用AES加密)：

加密后的数据使用 BLOB 类型存储，因为加密后的数据是二进制格式。



无忌教育官网 | wuji-edu.com

```
INSERT INTO students VALUES (23,  
AES_ENCRYPT('编程张无忌', 'key666'), 33);
```


student_id	student_name	age
1	(BLOB) 9 bytes	20
2	(BLOB) 6 bytes	21
3	(BLOB) 6 bytes	22
20	(BLOB) 6 bytes	33
21	(BLOB) 7 bytes	99
22	(BLOB) 7 bytes	34
23	(BLOB) 16 bytes	33
33	(BLOB) 16 bytes	33

2. 传输加密

- 使用 SSL/TLS 协议加密数据库连接（如MySQL的 `REQUIRE SSL` 选项）。

视图机制

-- 创建视图，仅暴露姓名和年龄

```
CREATE VIEW StudentView AS  
SELECT 姓名, 年龄 FROM Students;
```

-- 授予用户访问视图的权限

```
GRANT SELECT ON StudentView TO user_zwj;  
GRANT all privileges ON StudentView TO user_zwj;
```

防止SQL注入

原理：通过参数化查询避免拼接恶意SQL代码。

#使用参数化查询（安全）

```
execute("SELECT * FROM users WHERE username =  
%s", (user_input ))
```

数据备份与恢复

防止数据丢失。

```
#使用 mysqldump 备份整个数据库
mysqldump -u 用户名 -p 密码 --databases 数据库名
> backupfile.sql
```

#如果您只想备份单个表, 则请执行以下mysqldump命令:

```
mysqldump -u 用户名 -p 密码 数据库名 表名 >
backupfile.sql
```

审计与监控

- **日志审计:** 记录所有SQL操作 (如用户登录、数据修改) 。

```
#SQL Server启用审计
CREATE SERVER AUDIT SecurityAudit TO FILE
(FILEPATH='D:\AuditLogs\');

#MySQL
--安装审计插件

INSTALL PLUGIN audit_log SONAME
'audit_log.so';
-- 配置审计日志路径
SET GLOBAL audit_log_file =
'/var/log/mysql/audit.log';
```

【新拯救者】Ep11_数据库的安全性习题



在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep12_数据库的完整性

三大完整性

实体完整性

- 定义：确保每个表中的主键唯一且非空，唯一标识实体。
 - 主键值必须唯一且非空（NOT NULL）
 - 主键可以是单列或多列组合（复合主键）
- 实现方式：通过 PRIMARY KEY 约束定义

-- 列级主键

```
CREATE TABLE Student (  
    sno CHAR(9) PRIMARY KEY,  
    sname CHAR(20) NOT NULL  
);
```

-- 表级复合主键

```
CREATE TABLE SC (  
    sno CHAR(9),  
    cno CHAR(4),  
    PRIMARY KEY (sno, cno)  
);
```

参照完整性

- **定义：**维护表间外键关系，确保外键值有效或为 NULL
 - 外键值必须存在于被引用表的主键中，或为 NULL
 - **实现方式：**通过 FOREIGN KEY 定义

```
CREATE TABLE SC (  
    sno CHAR(9),  
    cno CHAR(4),  
    FOREIGN KEY (sno) REFERENCES  
Student(sno),  
    FOREIGN KEY (cno) REFERENCES Course(cno)  
);
```

用户定义完整性

- **定义：**根据业务规则自定义约束，如非空、唯一、检查条件。
 - 列值非空（`NOT NULL`）、唯一（`UNIQUE`）。
 - 列值满足特定条件（`CHECK`约束）。
 - 元组级约束（跨列条件限制）。
- **实现方式：**通过 `NOT NULL`、`UNIQUE`、`CHECK` 等约束定义。



无忌教育官网 | wuji-edu.com

```
create table stu(  
    sno char(9) primary key,  
    sex char(1) not null,  
    age int check(age>0)  
);
```

断言

定义更具一般性的约束,确保数据始终满足条件

`create assertion <断言名> <check子句> check子句`
的约束条件与 `WHERE` 子句的条件表达类似。

-- 确保所有员工的工资总和不超过100万元

```
CREATE ASSERTION total_salary_check
```

```
CHECK (
```

```
    (SELECT SUM(salary) FROM employees) <=
1000000
);
```

```
-----
drop assertion 断言名;
```

触发器

触发器也叫做 事件->条件->动作 规则。

当对一个表增 删 改时候, 对触发器里面的条件进行检查,

如果成立, 就执行触发器里面的动作

否则不执行里面的动作。

```
DELIMITER //      /*修改mysql的结束符, 将原来的分号(;)结束符修改为(//)*/
```

```
CREATE TRIGGER <触发器名>                                /*
每当触发事件发生时, UPDATE insert该触发器被激活*/
```

```
{BEFORE | AFTER} <触发事件> ON <表名> /*指明触发器激活的时间是在执行触发事件前或后*/
```

```
FOR EACH {ROW | STATEMENT}                                /*
定义触发器的类型
```

行级触发器 (FOR EACH ROW) 和语句级触发器 (FOR EACH STATEMENT)

```
BEGIN
    <trigger action body(触发动作体)>
/*仅当触发条件为真时才执行触发动作体*/
END
DELIMITER ;    /*改回结束符*/

-----

--
DELIMITER //
CREATE TRIGGER SC_T    /*SC_T是触发器的名字*/
AFTER UPDATE ON SC    /*UPDATE ON SC是触发事件*/
FOR EACH ROW    /*行级触发器。即每执行一次Grade的更新，下面规则就执行一次*/
BEGIN
    IF NewGrade >= 10    /*触发条件，只有条件为真时才执行*/
        INSERT INTO SC_U(Sno, Cno, Oldgrade, Newgrade)
            VALUES(Old.Sno, Old.Cno, Old.Grade, New.Grade);
    END IF;
END
DELIMITER ;
```

【新拯救者】Ep13_数据库的完整性习题

 在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2



无忌教育官网 | wuji-edu.com

【新拯救者】Ep14_关系数据理论(上)

常考计算题

- 1.求候选码
- 2.求最小函数依赖集
- 3.求模式分解
- 4.求满足第几范式

关系数据理论概述

- 解决数据冗余、插入/删除/更新异常问题

- 提供数据库设计的理论基础，指导表结构规范化
- **核心：**通过函数依赖和范式理论消除数据冗余和操作异常

关系模型的基本概念

术语	定义	示例
关系 (表)	数据按行和列存储的结构	学生表 (学号, 姓名, 年龄, 班级)
元组	表中的一行数据	(2027001, '张无忌', 30, '软件工程班')
属性	表中的列，对应字段名	学号、姓名、年龄
域	属性的取值范围 (如年龄为18-40的整数)	年龄 $\in [18, 40]$

函数依赖

定义：若属性集X的值唯一决定属性集Y的值，则称Y**函数依赖于**X，记作 $X \rightarrow Y$ 。

完全依赖与部分依赖

- **完全依赖：**Y 仅由整个候选码 X 决定 (如 学号 $\rightarrow$ 姓名 (学号唯一确定姓名))
- **部分依赖：**Y 依赖候选码的子集 (如 {学号,身份证} $\rightarrow$ 姓名 这就是部分函数依赖，需分解模式消除)

平凡依赖与非平凡依赖

- **平凡依赖**: $Y \subseteq X$ 时, $X \rightarrow Y$ 恒成立

{学号, 姓名, 性别, 年龄, 学号}, 显然学号可以决定它自身, 即 $\{\text{学号}\} \rightarrow \{\text{学号}\}$,
这就是一个平凡函数依赖

- **非平凡依赖**: 如果 $X \rightarrow Y$ 成立, 且 Y 不是 X 的子集, 那么称 $X \rightarrow Y$ 是非平凡函数依赖

假设每个学生的学号是唯一的, 那么 $\{\text{学号}\} \rightarrow \{\text{姓名, 性别, 年龄}\}$ 就是一个非平凡函数依赖,

因为学号能够决定学生的姓名、性别和年龄这些不同的属性, 且姓名、性别、年龄都不是学号的子集

求候选码

概念: 可以推导出所有属性的

如何选出候选码?

- 1) 只出现在左边的一定是候选码
- 2) 只出现在右边的一定不是候选码
- 3) 左右都出现的不一定
- 4) 左右都不出现的一定是候选码
- 5) 再求确定的候选码的闭包, 如果可以推出全部, 那么当前确定的就是候选码, 否则你要把每一个可能的值 放进当前确定的候选码里面进行求 闭包

$R \langle U, F \rangle$, $U(A, B, C, D, E, G)$, $F = \{AB \twoheadrightarrow C, CD \twoheadrightarrow E, E \twoheadrightarrow A, A \twoheadrightarrow G\}$, 求候选码:

B一定是候选码 D一定是候选码

G一定不是候选码

A不一定 C不一定 E不一定

$BD \twoheadrightarrow$ 啥也推不出来, 所以要把每一个可能的求闭包

$(BDA) +=$ 可退出 C E A G 所以可以退出 ABCDEG

$(BDC) +=$ 可退出 E A G 所以可以退出 ABCDEG

$(BDE) +=$ A G C 可以退出 ABCDEG

那么他的候选码最终是 $\{ (BDA), (BDC), (BDE) \}$;

超码: 能表示出所有属性的集合, 比如 $(BDA), (BDC), (BDE)$

BDCA BDEA ABCDE

候选码是最小的超码.

主码: 从候选码里面任意跳出一个 作为主码

主属性: 包含在所有候选码的属性 比如 ABCDE

非主属性: 不包含在候选码中的属性 , 上题为 G

全码: 所有的属性共同构成一个候选码时, 这时该候选码为全码

闭包计算:

属性集闭包: 给定属性集 X, 能推导出的所有属性集合。

--示例：若 $x = \{\text{学号}\}$ ，闭包 $x^+ = \{\text{学号}, \text{姓名}, \text{班级}\}$

Armstrong 公理系统

- 自反律：若 $Y \subseteq X$ ，则 $X \rightarrow Y$ 。
- 增广律：若 $X \rightarrow Y$ ，则 $XZ \rightarrow YZ$ 。
- 传递律：若 $X \rightarrow Y$ 且 $Y \rightarrow Z$ ，则 $X \rightarrow Z$ 。

范式理论

目标：消除数据冗余和操作异常。

第一范式 (1NF)

定义：表中所有属性不可再分（原子性）。

用户信息表							
编号	姓名	性别	年龄	联系电话	省份	城市	详细地址
1	张红欣	男	26	0378-23459876	河南	开封	朝阳区新华路23号
2	李四平	女	32	0751-65432584	广州	广东	白云区天明路148号
3	刘志国	男	21	0371-87659852	河南	郑州	二七区大学路198号
4	郭小明	女	27	0371-62556789	河南	郑州	新郑市薛店北街218号

如果系统经常会访问“地址”属性中的“城市”部分，那么就非要将“地址”这个属性重新拆分为省份、城市、详细地址等多个部分进行存储，这样在对地址中某一部分操作的时候将非常方便

修正：拆分为多行或多表。

第二范式 (2NF)

- 前提：满足1NF。
- 要求：消除非主属性对主键的部分依赖
- (在1NF的基础上进一步设计的。2NF要求每个非主键字段必须完全依赖于主键，而不能只依赖于主键的一部分)

```
-- 学生选课表 (主键: Sno, Cno)
-- 非主属性 '班级' 仅依赖 Sno, 导致部分依赖
CREATE TABLE SC (
    sno CHAR(9),
    cno CHAR(4),
    grade INT,
    class VARCHAR(20), -- 部分依赖 Sno
    PRIMARY KEY (sno, cno)
);
```

```
-- 学生表 (sno, class)
-- 选课表 (sno, cno, grade)
```

第三范式 (3NF)

- 前提：满足2NF。
- 要求：消除非主属性对主键的传递依赖

```
-- 学生表 (sno, sname, dno, dname)
      (Sno → Dno → Dname) dname 传递依赖于 sno
```

- **修正：**拆分为学生表 (sno, sname, dno) 和系表 (dno, dname)

BC范式 (BCNF)

- **前提：**满足3NF。
- **要求：**所有函数依赖的决定因素必须是**候选键**

```
-- 教师表 (tno, tname, cno)
      假设教师教的对应课程唯一      -- 候选码为 (tno, cno)

      -- 存在函数依赖 tno → cno 和 cno → tno , 但
      cno → tno 违反BCNF
```

- **修正：**拆分为教师表 (tno, tname) 和课程表 (cno, tno) 。

求最小函数依赖集

什么是依赖?

依赖是指关系中一个或一组属性的值可以决定其它属性的值

比如 $A \rightarrow B$ 这就是一个依赖

如何求最小依赖集?

- 1) 拆右边为多个元素的 比如 $A \rightarrow BC$ 拆为 $A \rightarrow B$ 和 $A \rightarrow C$
- 2) 除去当前元素, 求它的闭包, 把集合里面所有元素都弄完
- 3) 左边最小化 (通过遮住元素来看能不能退出其他元素)

比如 BCD , 遮住 B 看能退出 CD 吗, 遮住 C 能退出 BD 吗 遮住 D 看能退出 BC 吗

已知关系 $R \langle U, F \rangle$, $U \{A, B, C, D, E, F, G\}$ $F = \{BCD \rightarrow A, BC \rightarrow E, A \rightarrow F, F \rightarrow G, C \rightarrow D, A \rightarrow G\}$ 求 F 的最小依赖集

求模式分解

准则: 无损连接 和 保持函数依赖

无损: 就是分解后再次连接, 和分解之前一样

依赖: 依赖不变

依赖: 科系 $\rightarrow$ 系主任 学号 $\rightarrow$ 科系

学号	科系	系主任
1	电气	王主任
2	计算机	李主任

学号
1
2
科系
电气
计算机
系主任
王主任
李主任

已知 $R(ABCDEFGH)$, $F=\{A \rightarrow D, E \rightarrow D, D \rightarrow B, BC \rightarrow D, DC \rightarrow A\}$, 求保持函数依赖的3NF的分解

- 1) 求出最小函数依赖集
- 2) 把不在 F 里面的属性都找出来, 单独分一类
- 3) 把每一个依赖左边相同的分成一类, 没有一样的, 那么就把 $A \rightarrow D$ 改为 $\{AD\}$, 如果一样 $\{A \rightarrow B, A \rightarrow C\}$ 那么改为 $\{ABC\}$
- 4) 如果候选码没出现在分离里面, 把任意一个候选码作为一类

最小函数依赖集 $F_{min}=\{A \rightarrow D, E \rightarrow D, D \rightarrow B, BC \rightarrow D, DC \rightarrow A\}$

GH 没在 F 里面, 单独一类 $\{GH\}$

候选码: CE

结果: $\{AD\}\{ED\}\{DB\}\{BCD\}\{DCA\}\{CE\}\{GH\}$

【新拯救者】Ep15_关系数据理论(下)

求满足第几范式?

1NF=有部分依赖

候选键: AB

若 $B \rightarrow C$ 则 C 部分依赖于 AB

2NF=有传递依赖

同时
满足

$B \rightarrow A$

$A \rightarrow C$

$A \rightarrow B$

BCNF=左边全是候选键

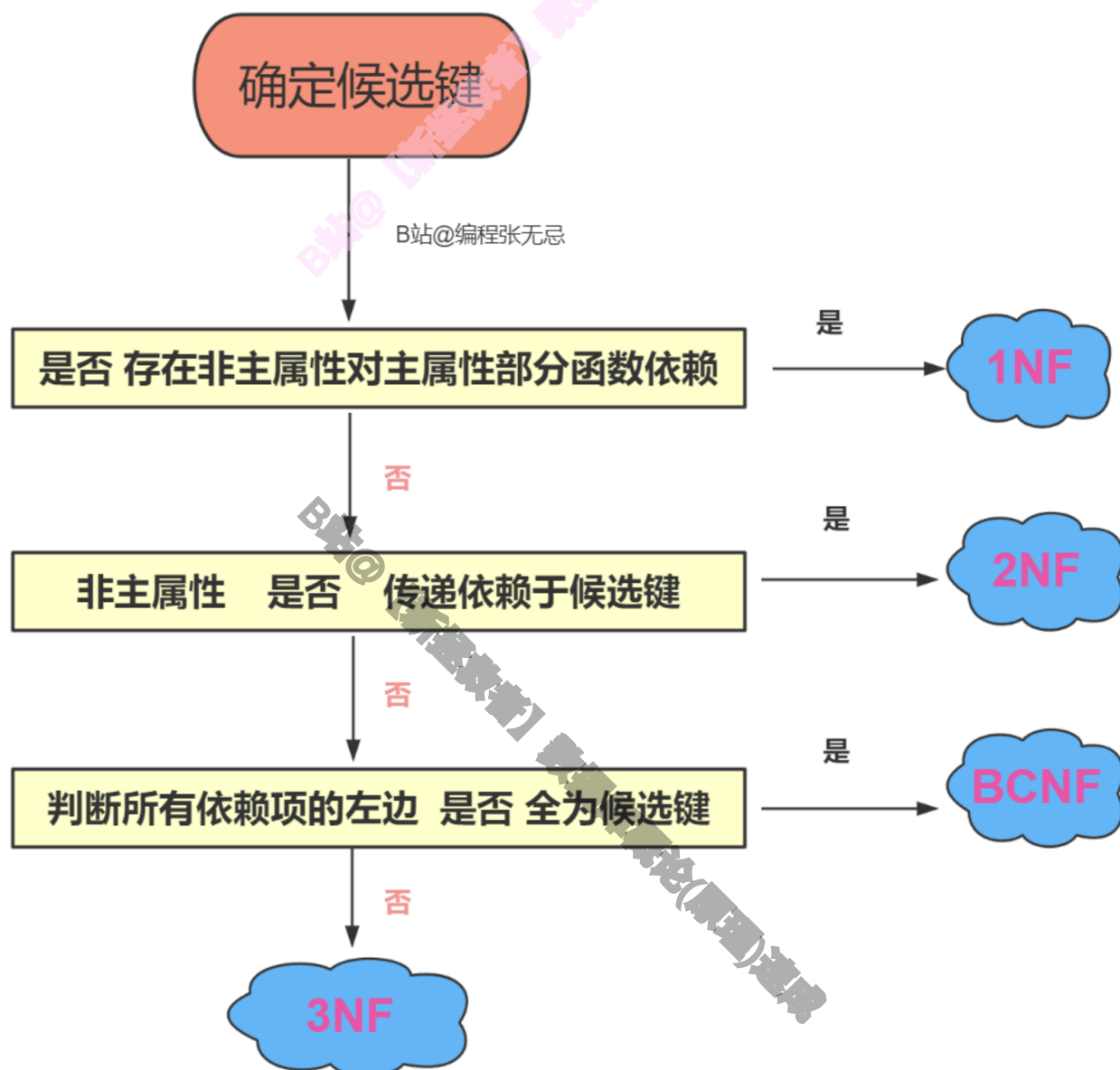
$F = \{ A \rightarrow B, B \rightarrow A, A \rightarrow C \}$,

若 A, B 都是候选键, 则满足

3NF=左边不全是候选键

上面都不满足, 就是3NF啦

如何解题



题目解析

1. $R(A, B, C)$, $F=\{A \rightarrow B, B \rightarrow A, A \rightarrow C\}$, 问满足第几范式

L: LR: A,B 非主属性是C

是BCNF,解析看视频

2. $R(A, B, C, D)$, $F=\{B \rightarrow D, D \rightarrow B, AB \rightarrow C\}$, 问满足第几范式

$L=A$ LR: B, D 非主属性是 C 是 3NF

【新拯救者】Ep16_关系数据理论习题

 在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep17_数据库设计

常考计算题

1. ER图转关系模型
2. ER图构建

数据库设计的基本步骤

需求分析

概念结构设计 ER图或者是设计数据字典

逻辑结构设计 把ER图转为逻辑模型

物理结构设计 把逻辑模型转为物理模型

数据库实施 写SQL代码

数据库运行维护 性能检测

ER图

实体是长方形体现, 属性则是椭圆形, 关系为菱形

1对1 (1:1) : 实体A中的一个实例 唯一对应 实体B中的一个实例

一个人有一个身份证号, 一个身份证也只对应一个人

1对多 (1:N) : 实体A中的一个实例 对应 实体B中的多个实例, 但实体B中的一个实例 只能对应 实体A中的一个实例

一个 部门 有多个 员工, 但一个 员工 只属于一个 部门。

多对多 (M:N) : 实体A中的一个实例对应 实体B中的多个实例

一个 Student 可选多门 课, 一门 课 可被多个 Student 选修。

ER图转关系模型



1:1

B站@【拯救者】数据库概论(原理)速成

要把关系合并到任意一个表,并且把另外的表里面的主键放到另一个里面



1:n

把A的主码放到B里, 把关系的属性放到B里



m:n

需要两边实体的主键和自身的属性,单独作为一个关系模式

一个图书借阅管理数据库要求提供下述服务:

1) 可随时查询书库中现有书籍的品种、数量与存放位置。所有各类书籍均可由书号唯一标识

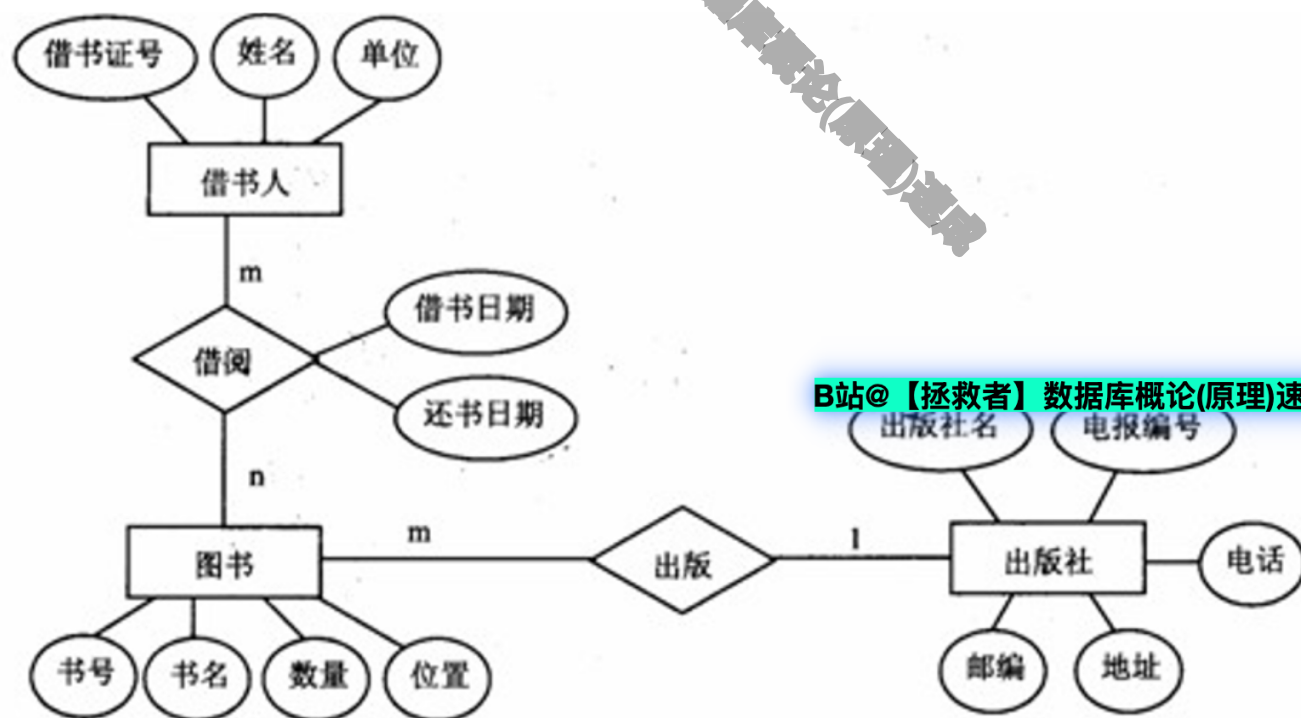
2) 可随时查询书籍借还情况, 包括借书人单位、姓名、借书证号、借书日期和还书日期

约定: 任何人可借多种书, 任何一种书可为多个人所借, 借书证号具有唯一性

3) 当需要时, 可通过数据库中保存的出版社的电报编号、电话、邮编及地址等信息向相应出版社增购有关书籍。

约定: 一个出版社可出版多种书籍, 同一本书仅为一个出版社出版, 出版社名具有唯一性

根据以上描述, 建立满足本需求的E-R图, 和转为的关系模型



B站@【拯救者】数据库概论(原理)速成

借书人(借书证号,姓名 单位) 图书(书号,书名,数量,位置) 借书(借书证号,书号,借书日期,还书日期)
出版社(出版社名,电报编号,邮编,电话,地址)

出版社名

【新拯救者】Ep18_数据库设计习题



在线练习 请点击

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

【新拯救者】Ep19_数据库编程

嵌入式SQL

将SQL语句嵌入到高级程序设计语言（如C、Java）中，借助宿主语言的控制功能实现过程化编程。

嵌入式SQL的处理过程

预编译转换为函数调用

主语言编译

变成主语言所编译的类型

SQL与主语言的通信

SQL给主语言传递状态

数据库系统通过SQLCA向主语言返回SQL语句的执行状态, 包括错误代码、警告信息等。

SQLCODE: 执行结果代码 (0表示成功, 负数表示错误, 正数表示警告)

```
EXEC SQL CONNECT TO schooldb USER 'user'
PASSWORD 'password';

if (sqlca.sqlcode != 0) {
    printf("连接失败: %s\n",
sqlca.sqlerrm.sqlerrmc);
    return 1;
}

EXEC SQL SELECT name INTO :student_name FROM
students WHERE id = :student_id;

if (sqlca.sqlcode == 0) {
    printf("学生姓名: %s\n", student_name);
}
```



```
    } else {  
        printf("查询失败: %s\n",  
sqlca.sqlerrm.sqlerrmc);  
    }
```

主语言给SQL提供参数

主语言变量通过占位符（如:variable）与SQL语句交互。在JDBC中通过PreparedStatement设置参数，防止SQL注入

```
String url =  
"jdbc:mysql://localhost:3306/schooldb";  
String user = "user";  
String password = "password";  
  
try (Connection conn =  
DriverManager.getConnection(url, user, password)) {  
    // 参数化查询  
    String sql = "SELECT name FROM students  
WHERE id = ?";  
    PreparedStatement pstmt =  
conn.prepareStatement(sql);  
    pstmt.setInt(1, 101); // 设置参数值  
  
    ResultSet rs = pstmt.executeQuery();  
    if (rs.next()) {  
        System.out.println("学生姓名: " +  
rs.getString("name"));  
    }  
}
```

```
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

SQL把查询结果交给主语言处理(游标,和主变量实现)

游标 (Cursor) : 用于遍历多行查询结果集。

主变量映射: 将查询结果赋值到主语言变量中。

```
try (Connection conn =  
DriverManager.getConnection(url, user, password)) {  
    Statement stmt = conn.createStatement();  
    ResultSet rs = stmt.executeQuery("SELECT id,  
name FROM students");  
    while (rs.next()) {  
        int id = rs.getInt("id");  
        String name = rs.getString("name");  
        System.out.println("ID: " + id + ", 姓名: "  
+ name);  
    }  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

【新拯救者】Ep20_关系查询 处理和查询优化

查询过程



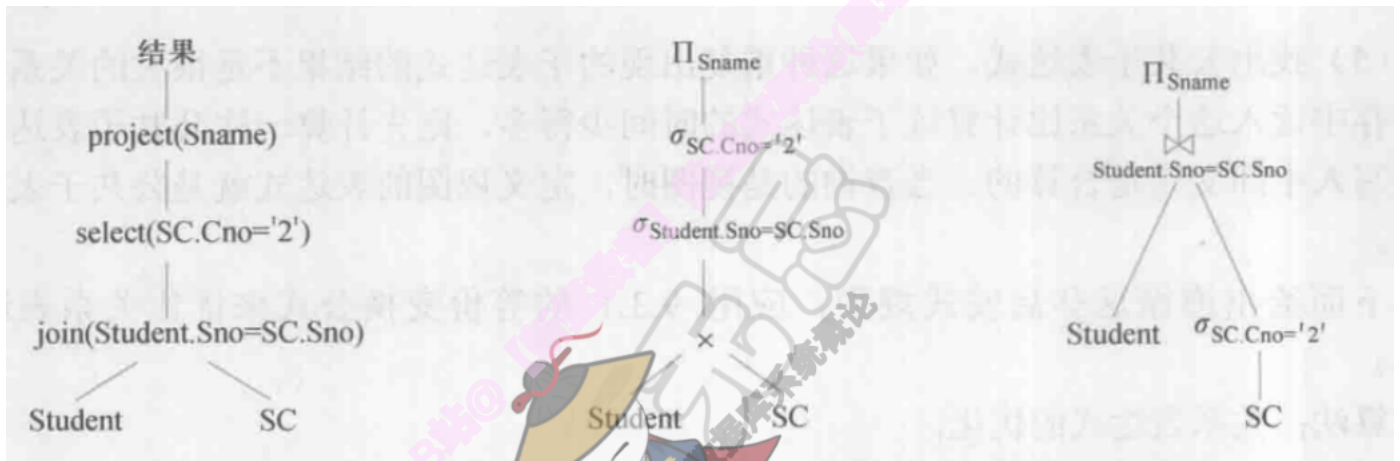
查询优化的四个阶段:

查询分析

查询检查

查询优化 (包括代数优化, 物理优化)

查询执行



选择运算尽量先做

把投影运算和选择运算同时执行

把投影同它前后的双目运算符连接起来

【新拯救者】Ep21_数据恢复技术

事务的四大特性:

A: 原子性 atomic

要么全做, 要么全不做

C:一致性 consistent

一致性与原子性密切相关, 要么全做要么全不做, 否则就会造成数据不一致。

比如: 银行汇钱, 一个减少100, 另一个一定要增加100

I: 隔离性isolate

一个事务的执行不能被其他事务所干扰

D: 持久性 duration

数据库的改变是永久的。比如要落入磁盘

数据恢复技术核心目标

确保数据库在发生**系统崩溃、事务故障**或**介质故障**后,能够恢复到一致的状态,保障数据的**持久性**(ACID中的D)。

故障的种类:

1、事物内部故障

采取REDO重做和UNDO撤销技术

2、系统故障 DBMS

系统突然停转, 系统要重启

3、介质故障

硬件损坏

4、计算机病毒

恢复方式

数据转储

对失败的事务重新执行

日志文件

记录事务对数据的更新操作的文件

-- UNDO日志 撤销未提交的事务 (通过UNDO日志)

-- REDO日志 重做已提交但未持久化的事务 (通过REDO日志)

-- 事务T1: 向账户A转账100元

```
BEGIN TRANSACTION;
UPDATE Account SET Balance = Balance - 100 WHERE
AccNo = 'A';
UPDATE Account SET Balance = Balance + 100 WHERE
AccNo = 'B';
COMMIT;
```

日志记录

```
<T1, BEGIN>
<T1, A, -100>
<T1, B, +100>
<T1, COMMIT>
```

- 事务T1正在更新账户A和B, 系统在提交前崩溃。
- 日志记录:

```
<T1, BEGIN>
<T1, A, -100>
<T1, B, +100>
```

恢复过程：

1. **前滚**：无已提交的REDO操作（T1未提交）。
2. **回滚**：通过UNDO日志撤销T1的修改，恢复A和B的原始值。

恢复策略

事务故障的恢复

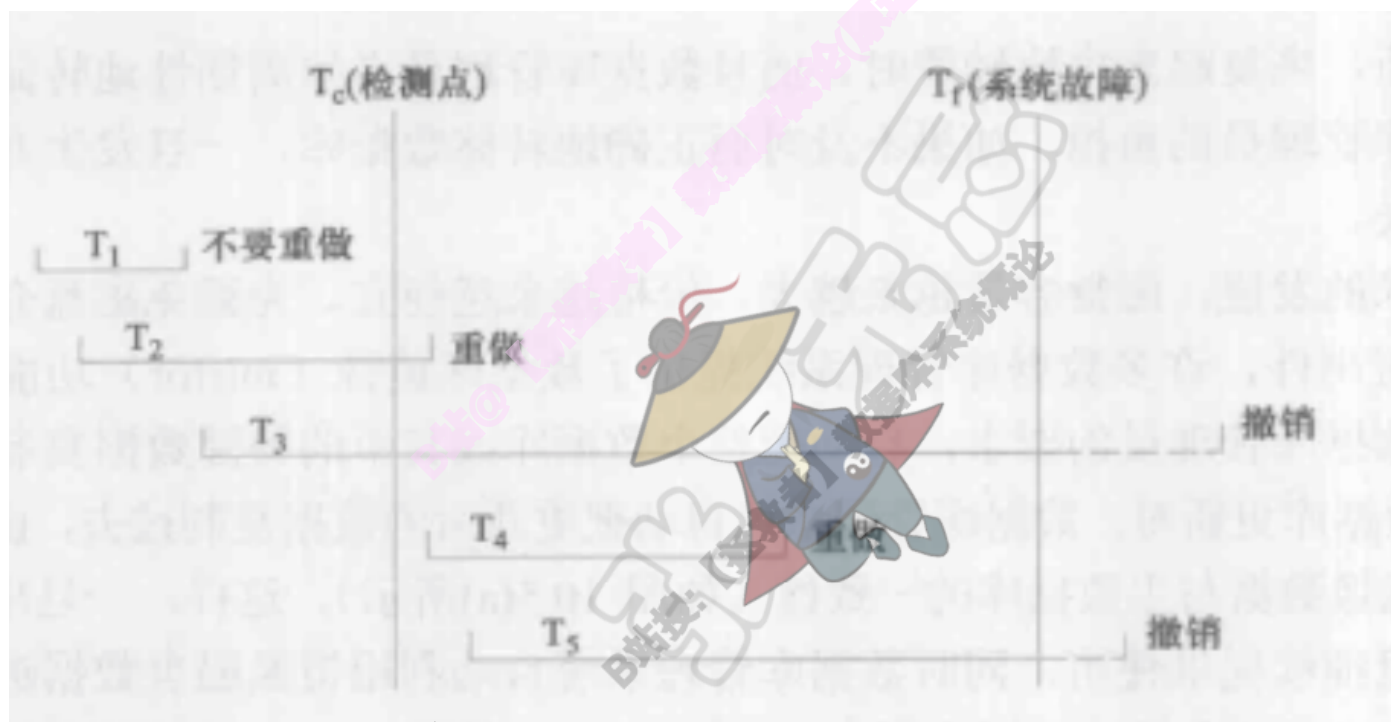
事务异常终止，那么就撤销之前的所有操作

系统故障的恢复

还没执行完的事务UNDO、丢失的事务REDO

介质故障的恢复

重装数据库，重做已经完成的事务



【新拯救者】Ep22_并发控制

确保多个事务并发执行时,数据库的一致性和隔离性,避免数据异常(如脏读、不可重复读、幻读)

事务的隔离级别

隔离级别	脏读 (Dirty Read)	不可重复读 (Non-repeatable Read)	幻读 (Phantom Read)	实现方式
Read Uncommitted	✓ 允许	✓ 允许	✓ 允许	无锁
Read Committed	✗ 禁止	✓ 允许	✓ 允许	行级锁 (短锁)
Repeatable Read	✗ 禁止	✗ 禁止	✓ 允许	多版本并发控制 (MVCC)
Serializable	✗ 禁止	✗ 禁止	✗ 禁止	严格的锁或时间戳排序

并发带来的问题

1. 丢失修改

两个或多个事务并发修改同一数据时, 其中一个事务的修改被另一个事务的更新覆盖, 导致部分修改丢失

事务A和事务B同时读取同一数据。

事务A修改数据后写回, 事务B也修改数据并写回, 覆盖了事务A的修改。

2. 脏读

一个事务读取了另一个●●未提交事务●●修改的数据。如果未提交的事务回滚, 读到的数据就是无效的 (“脏数据”)

小明向小红转账。

事务A (小明): `UPDATE 账户 SET 余额 = 余额 - 100 WHERE 姓名='小红';` (未提交)

事务B (小红): `SELECT 余额 FROM 账户 WHERE 姓名='小红';` → 看到余额多了100元。

结果: 事务A回滚 (撤销转账), 但小红已读到“脏数据” (实际余额未增加)

3. 不可重复读

在同一事务内, 多次读取同一数据, 结果不同。因为其他事务在两次读取之间●●提交●●了修改。

小明查看商品库存。

事务A (小明): 第一次查询 `SELECT 库存 FROM 商品 WHERE 名称='手机';` → 库存为10。

事务B (商家): `UPDATE 商品 SET 库存 = 9 WHERE 名称='手机';` (提交)。

事务A再次查询 → 库存变为9 (同一事务内两次读取结果不同)

4. 幻读

在同一事务内，同样的查询条件返回了 不同数量的行 。因为其他事务插入了符合查询条件的新数据并提交。

小明统计某类商品种类。

事务A（小明）：第一次查询 `SELECT COUNT(*) FROM 商品 WHERE 类别='电子产品'`；→ 5种。

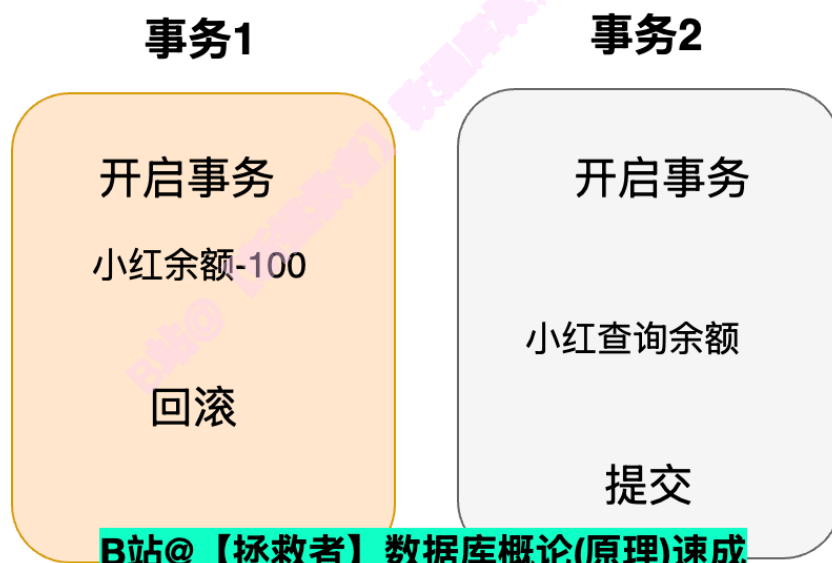
事务B（供应商）：插入一条新电子产品并提交。

事务A再次查询 → 结果为6种。

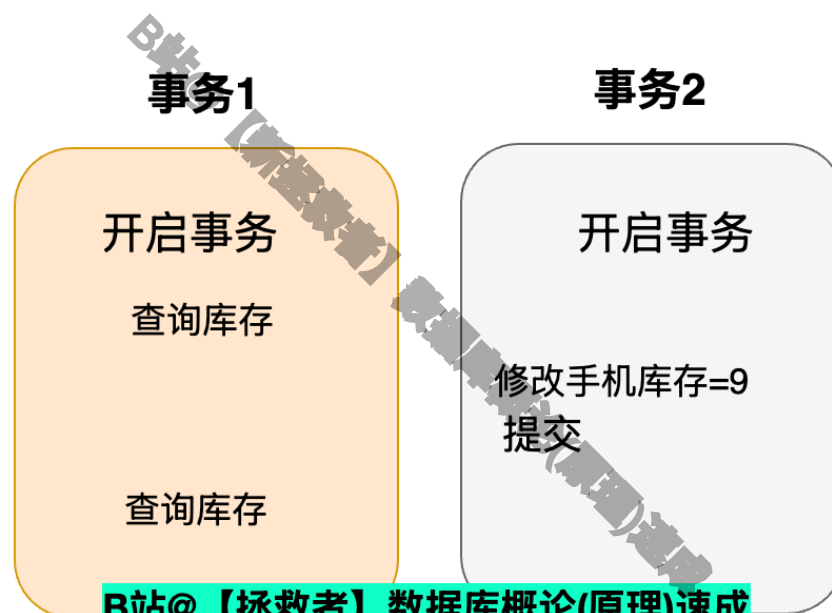
结果：同一查询条件返回了不同行数（多了1条“幻影”数据）。



B站@【拯救者】数据库概论(原理)速成



B站@【拯救者】数据库概论(原理)速成



B站@【拯救者】数据库概论(原理)速成



锁机制

通过锁定数据资源，控制事务的并发访问。

共享锁（S锁）：允许多个事务同时读取同一数据，但阻止其他事务获取排他锁

```
LOCK TABLE Account IN SHARE MODE; -- 共享锁
```

排他锁（X锁）：只允许一个事务独占数据，其他事务无论是读还是写都无法获取锁，直到排他锁被释放。

```
LOCK TABLE Account IN EXCLUSIVE MODE; -- 排他锁
```

封锁协议

1、一级封锁协议

修改时，必须加x锁，直到结束

2、二级封锁协议

读的时候，加s锁，用完就放

3、三级封锁协议

读的时候，加s锁，直到结束

一级锁协议：解决丢失修改

二级锁协议：解决读脏数据

三级锁协议：解决不可重复读

T ₁	T ₂	T ₁	T ₂	T ₁	T ₂
① Xlock A		① Slock A		② Xlock C	
② R(A)=16		Slock B		R(C)=100	
③	Xlock A	R(A)=50		C=C*2	
④ A=A-1	等待	R(B)=100		W(C)=200	
W(A)=15	等待	A+B=150		②	Slock C
Commit	等待	②	Xlock B	等待	等待
Unlock A	等待	等待	等待	③ ROLLBACK	等待
⑤	获得Xlock A	等待	等待	(C恢复为100)	等待
	R(A)=15	③ R(A)=50	等待	Unlock C	等待
	A=A-1	R(B)=100	等待	④	获得Slock C
⑥	W(A)=14	A+B=150	等待		R(C)=100
	Commit	Commit	等待	⑤	Commit
	Unlock A	Unlock A	等待		Unlock C
		Unlock B	等待		
		④	获得Xlock B		
			R(B)=100		
			B=B*2		
			W(B)=200		
		⑤	Commit		
			Unlock B		
(a) 没有丢失修改		(b) 可重复读		(c) 不读“脏”数据	

事务T在修改数据R之前必须先对其加X锁，直到事务结束才释放，这是（A）

- A. 一级封锁协议 B. 二级封锁协议
C. 三级封锁协议 D. 零级封锁协议

死锁

- 事务T1：锁定账户A → 请求锁定账户B。
- 事务T2：锁定账户B → 请求锁定账户A。

- **结果：** T1和T2互相等待对方释放锁，形成死锁。

本质是：事务故障

设事务T1和T2，对数据库中地数据A进行操作，可能有如下几种情况，请问哪一种不会发生冲突操作（D）

- A. T1正在写A， T2要读A
- B. T1正在写A， T2也要写A
- C. T1正在读A， T2要写A
- D. T1正在读A， T2也要读A

https://appb9cmdvkz7101.h5.xiaoeknow.com/v1/evaluation_wechat/practice/wb_course_6802cc49bc878_xnjHZ94N?type=2

B站@【新源教育】
资源更新(刷题)进群